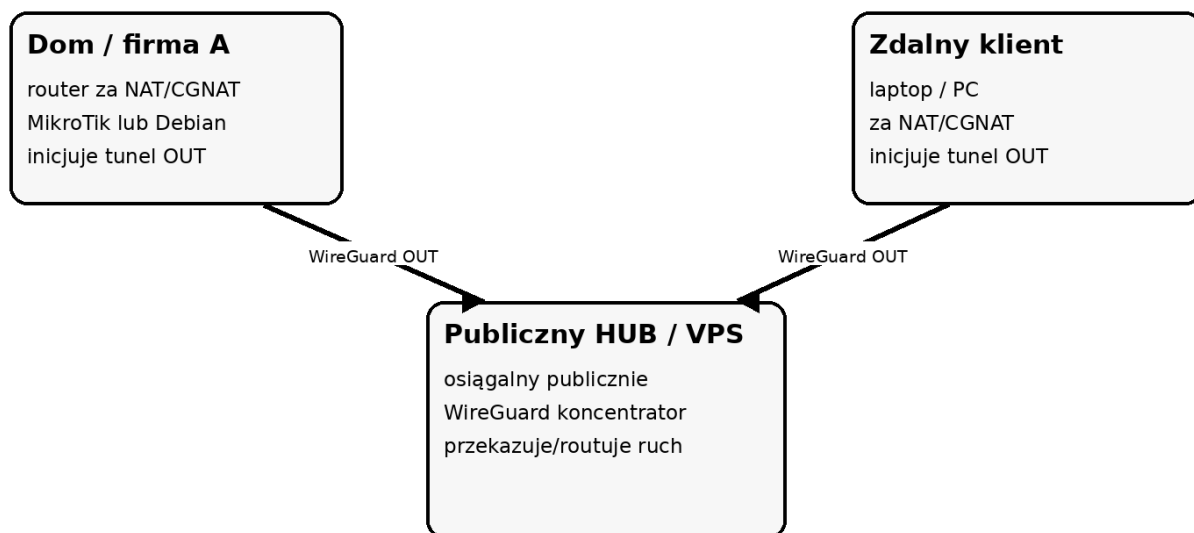


Dzień 22 - CGNAT, WireGuard HUB, Tailscale, DNS i MTU

Jak połączyć się z routerem/MikroTikiem lub siecią bez własnego publicznego IP

Dokumentacja obejmuje wykonane laboratorium oraz praktyczny poradnik dla przypadku, gdy domowy router lub MikroTik jest za NAT/CGNAT. Klucze prywatne są celowo pominięte.

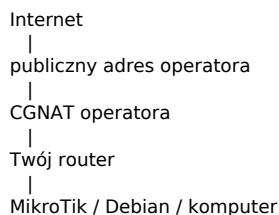
Jak połączyć dwie strony bez publicznego IP



Model: obie końcówki mogą być bez publicznego IP; publicznie osiągalny jest tylko koncentrator/HUB.

1. Co oznacza CGNAT

CGNAT (Carrier-Grade NAT) to NAT wykonywany przez operatora. Użytkownik może dostać na WAN adres ze współdzielonej przestrzeni 100.64.0.0/10, a właściwy publiczny IPv4 znajduje się po stronie operatora. W takim układzie zwykłe przekierowanie portu na domowym routerze zwykle nie wystarczy, bo nie kontrolujemy NAT-u operatora.



Zakres 100.64.0.0/10 jest oficjalnie przeznaczony jako Shared Address Space.

2. Jak sprawdzić, czy masz CGNAT

Porównujemy adres WAN pokazany przez router z publicznym adresem widzianym z Internetu. W naszym teście D-Link pokazał `85.28.166.100`, a `tailscale netcheck` również raportował `85.28.166.100`. To wskazuje, że w chwili testu router miał publiczny IPv4 bezpośrednio na WAN, a nie CGNAT.

The screenshot shows the 'WAN' configuration page of a D-Link router. It displays the 'Dynamic IPv4' connection type, which is 'Connected'. The 'IP address' field is highlighted with a black box and contains the text 'WAN IPv4 = 85.28.166.100'. Other fields include 'Subnet mask' (255.255.255.0) and 'Gateway IP address' (85.28.166.1). Buttons for 'CHANGE CONFIGURATION' and 'ADVANCED MODE' are visible at the bottom.

Rzeczywisty zrzut z panelu D-Link: WAN IPv4 = 85.28.166.100.

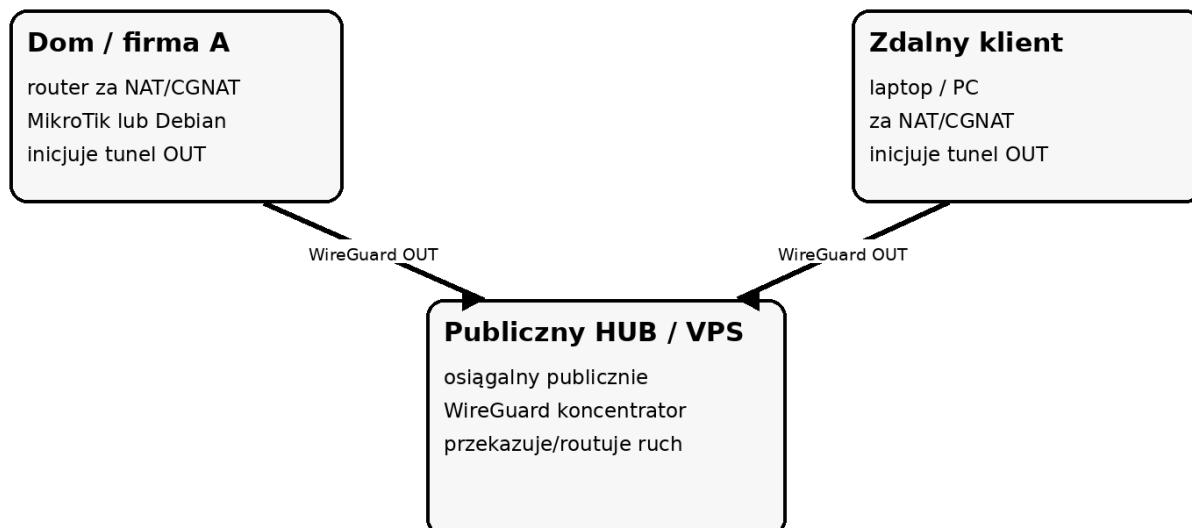
Przykład CGNAT:
WAN routera: 100.100.20.5
publiczny IP w Internecie: 85.x.x.x
=> podejrzenie CGNAT

Nasz test:
WAN routera: 85.28.166.100
tailscale netcheck IPv4: 85.28.166.100
=> publiczny IPv4 na WAN

3. Połączenie bez publicznego IP - wariant z HUB-em/VPS

Jeżeli domowy MikroTik lub Debian nie ma publicznego IP, używamy osiągalnego serwera pośredniego - koncentratora WireGuard. Najczęściej jest to VPS z publicznym IPv4. Urządzenia za NAT/CGNAT nie czekają na połączenie przychodzące; same inicjują połączenie wychodzące do HUB-a.

Jak połączyć dwie strony bez publicznego IP



Kluczowa zasada: publiczny adres musi mieć koncentrator, ale nie muszą go mieć obie końcówki.

4. Nasza lokalna symulacja koncentratora

Zamiast kupować VPS, zasymulowaliśmy osiągalny HUB lokalnie. Debian pełnił rolę koncentratora `10.80.0.1/24`, a MikroTik rolę klienta `10.80.0.2/24`.

Debian HUB:
wg-hub = 10.80.0.1/24
UDP 51830

MikroTik:
wg-hub-client = 10.80.0.2/24
listen-port 13232
endpoint HUB-a w labie = 192.168.122.1:51830

5. Koncentrator na Debianie - wykonane polecenia

```
umask 077
wg genkey | tee ~/wg-hub-private.key | wg pubkey > ~/wg-hub-public.key

sudo ip link add dev wg-hub type wireguard
sudo ip address add 10.80.0.1/24 dev wg-hub
sudo wg set wg-hub private-key ~/wg-hub-private.key listen-port 51830
sudo ip link set wg-hub up
sudo wg show wg-hub
```

Klucza prywatnego nie publikujemy ani nie zapisujemy w dokumentacji portfolio.

6. MikroTik jako klient HUB-a

```
/interface wireguard add name=wg-hub-client listen-port=13232
/ip address add address=10.80.0.2/24 interface=wg-hub-client

/interface wireguard peers add \
interface=wg-hub-client \
public-key="<PUBLIC_KEY_HUB>" \
endpoint-address=192.168.122.1 \
endpoint-port=51830 \
allowed-address=10.80.0.1/32 \
persistent-keepalive=25
```

W realnym wariancie z VPS `endpoint-address` byłby publicznym adresem IP lub nazwą DNS serwera VPS.

7. Dodanie MikroTika jako peera na HUB-ie

```
sudo wg set wg-hub \
peer "<PUBLIC_KEY_MIKROTIK>" \
allowed-ips 10.80.0.2/32
```

```
sudo wg show wg-hub
```

Rzeczywisty wynik: Debian zobaczył endpoint `192.168.122.75:13232`, świeży handshake oraz transfer `180 B received, 92 B sent` w obserwowanym momencie.

```
/ping 10.80.0.1 count=4
```

```
sent=4 received=4 packet-loss=0%
min-rtt=561us avg-rtt=667us max-rtt=765us
```

8. Jak podłączyć zwykły router bez WireGuard

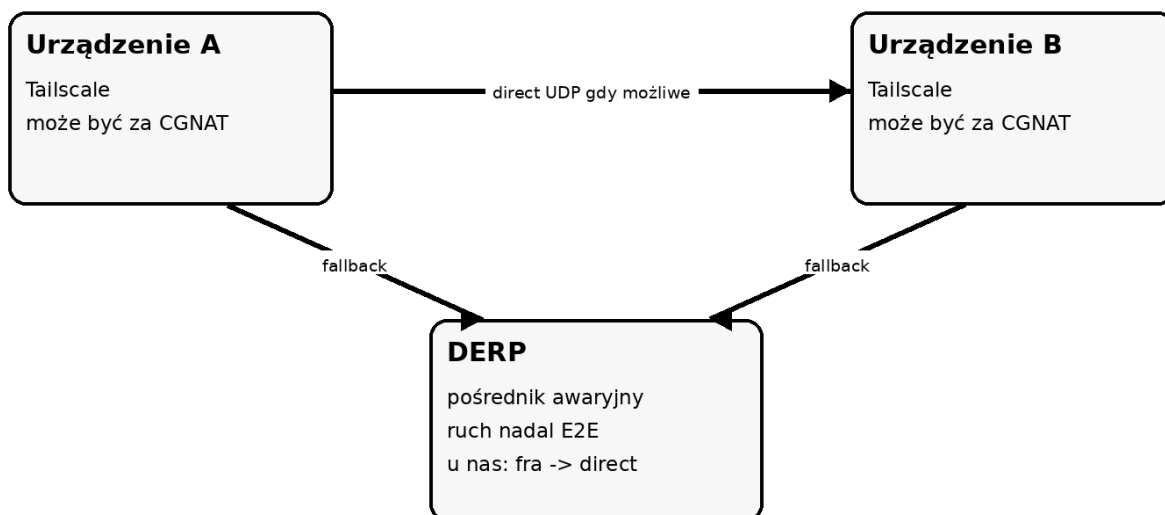
Zwykły router nie musi sam obsługiwać WireGuard. Za routerem może działać Debian, Raspberry Pi lub mały serwer, który zestawia tunel wychodzący do HUB-a. Jeżeli chcemy zdalnie docierać do całej sieci LAN, taki Debian musi działać jako router między VPN i LAN: wymagane jest IP forwarding oraz odpowiedni routing/firewall; w zależności od projektu może być potrzebny NAT.

```
Internet / CGNAT
|
zwykły router
|
Debian / mały serwer
WireGuard client -> publiczny HUB
|
LAN 192.168.x.0/24
```

Jeżeli router ma natywny WireGuard, może pełnić tę rolę sam. MikroTik jest wygodny, bo można na nim jednocześnie skonfigurować VPN, trasy, VLAN-y i firewall.

9. Tailscale - prostszy wariant bez publicznego IP

Tailscale: direct, a gdy się nie da - relay



Tailscale wykorzystuje WireGuard i automatyzuje odnajdywanie peerów oraz NAT traversal. Próbuje połączenia direct, a gdy bezpośrednia ścieżka nie jest możliwa, używa relaya.

```
tailscale status
tailscale ping 100.99.239.29
tailscale netcheck
```

W naszym teście `tailscale ping` początkowo szedł przez `DERP(fra)`, a następnie przeszedł na połączenie bezpośrednie `via 158.180.18.82:41641`.

DERP przekazuje zaszyfrowane pakiety. Połączenie pozostaje end-to-end szyfrowane WireGuardem; różnica między direct i relay dotyczy głównie ścieżki i wydajności.

10. Tailscale subnet router - dostęp do urządzeń bez Tailscale

Jeżeli Tailscale jest tylko na Debianie, a chcesz zdalnie dojść także do innych urządzeń w domowym LAN, Debian może zostać subnet routerem. To praktyczny wariant dla zwykłego routera bez funkcji VPN.

```
# przykład - NIE wykonywaliśmy tego jeszcze w Dniu 22
echo 'net.ipv4.ip_forward = 1' | sudo tee -a /etc/sysctl.d/99-tailscale.conf
sudo sysctl -p /etc/sysctl.d/99-tailscale.conf
```

```
sudo tailscale set --advertise-routes=192.168.0.0/24
```

Po reklamowaniu trasy trzeba ją zatwierdzić w panelu administracyjnym Tailscale. Ten wariant jest tylko opisany - nie oznaczamy go jako wykonany.

11. Split tunnel i full tunnel

Split tunnel vs full tunnel

SPLIT TUNNEL

AllowedIPs = 192.168.10.0/24

tylko VLAN10 idzie przez VPN

Internet idzie normalną trasą

FULL TUNNEL

AllowedIPs = 0.0.0.0/0

cały IPv4 idzie przez VPN

serwer musi routować i zwykle NATować

Split tunnel kieruje przez VPN tylko wybrane podsieci, np. `192.168.10.0/24`. Full tunnel dla IPv4 używa `0.0.0.0/0`, ale wtedy serwer VPN musi routować ruch i zwykle wykonywać NAT, aby klient mógł wychodzić do Internetu przez serwer.

12. Diagnostyka DNS

Jeżeli ping po adresie IP działa, ale nazwy nie działają, najpierw podejrzewamy DNS.

```
# Windows
ping 1.1.1.1
nslookup example.com
ipconfig /all
```

Schemat: IP działa + nazwa nie działa -> sprawdź DNS.

13. Diagnostyka MTU

Test MTU wykonany na Windows

```
ping 1.1.1.1 -f -l 1472 -> 4/4, 0% strat

1472 + 28 bajtów IPv4+ICMP = 1500

ping 1.1.1.1 -f -l 1473 -> fragmentacja wymagana

Wniosek: zwykła ścieżka Windows -> Internet ma MTU 1500

Uwaga: to nie był pomiar samego tunelu WireGuard.
```

Wykonaliśmy realny test na Windows. `1472` bajty danych ICMP przeszły, `1473` już wymagało fragmentacji przy ustawionym DF. Dla IPv4+ICMP dodajemy 28 bajtów, więc `1472 + 28 = 1500`.

Uwaga: był to pomiar zwykłej ścieżki Windows -> Internet, nie samego tunelu WireGuard. WireGuard ma dodatkowy narzut, dlatego efektywne MTU tunelu może być niższe.

14. Diagnostyka VPN - kolejność

1. Czy Endpoint jest poprawny?
2. Czy UDP dociera do serwera / HUB-a?
3. Czy firewall przepuszcza UDP?
4. Czy PublicKey peerów są poprawne?
5. Czy jest latest handshake?
6. Czy AllowedIPs zawiera właściwe sieci?
7. Czy routing zna drogę do celu?
8. Czy firewall FORWARD przepuszcza ruch?
9. Jeśli IP działa, ale nazwy nie -> DNS.
10. Jeśli większy ruch się zaczyna -> MTU.

15. Nasz rzeczywisty tailscale netcheck

Pole	Wynik	Interpretacja
UDP	true	UDP wychodzący działa
IPv4	85.28.166.100:40255	publiczny adres/port widziany przez STUN
IPv6	no	brak działającego IPv6 w tym teście
MappingVariesByDestIP	false	mapowanie nie zmieniało się zależnie od celu
PortMapping	UPnP, NAT-PMP	router wspiera automatyczne mapowanie
Nearest DERP	Warsaw - 8.7 ms	najbliższy relay wg testu

16. Co zrobiliśmy praktycznie

Element	Stan
Uruchomienie wg-prod po restarcie	WYKONANE
Utworzenie osobnego wg-hub na Debianie	WYKONANE
MikroTik wg-hub-client	WYKONANE
Handshake MikroTik <-> Debian HUB	WYKONANE
Ping MikroTik -> HUB 10.80.0.1	4/4, 0% strat
Split/full tunnel - rozróżnienie	PRZECWICZONE
Tailscale status/ping/netcheck	WYKONANE
DERP -> direct w jednym teście	ZAOBSERWOWANE
Sprawdzenie WAN routera	publiczny IPv4, brak CGNAT w chwili testu
Test MTU Windows	1500 dla zwykłej ścieżki
Tailscale subnet router	TYLKO OMÓWIONE - nie wykonane

17. Najważniejsze rzeczy do zapamiętania

CGNAT -> port forwarding u klienta zwykle nie wystarczy.
Bez publicznego IP -> tunel OUT do publicznego HUB-a.
Tailscale -> bez ręcznego publicznego endpointu po stronie użytkownika.
DERP -> relay, ale ruch nadal end-to-end szyfrowany.
Split tunnel -> wybrane podsieci przez VPN.
Full tunnel -> 0.0.0.0/0 + routing/NAT po stronie wyjścia.
IP działa, nazwy nie -> DNS.
Większy ruch/usługi się zacinają -> sprawdź MTU.

18. Źródła aktualnej dokumentacji

RFC 6598 - Shared Address Space 100.64.0.0/10: <https://www.rfc-editor.org/info/rfc6598/>

Tailscale - Device connectivity: <https://tailscale.com/docs/reference/device-connectivity>

Tailscale - DERP servers: <https://tailscale.com/docs/reference/derp-servers>

Tailscale - Configure a subnet router: <https://tailscale.com/docs/features/subnet-routers/how-to/setup>

Tailscale - Exit nodes: <https://tailscale.com/docs/features/exit-nodes>

MikroTik RouterOS - WireGuard: <https://help.mikrotik.com/docs/spaces/ROS/pages/69664792/WireGuard>